

جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

SE444 - Introduction to Artificial Intelligence

Lecture 5: Adversarial Search and Games 🎮

Prof. Anis Koubaa | akoubaa@alfaisal.edu

This lecture dives deep into the fascinating world of **adversarial search**, where AI agents compete in dynamic, multi-agent environments. You'll explore the fundamental principles of **game theory** as we uncover how AI strategizes in two-player games, aiming for optimal outcomes. We will master techniques like the **Minimax algorithm** for decision-making under uncertainty and learn how **alpha-beta pruning** significantly enhances its efficiency. Finally, we'll discuss practical applications, from classic board games to modern AI achievements like **Deep Blue**, equipping you with the skills to develop intelligent agents capable of navigating complex competitive scenarios.

Course Outline 📋

01

Adversarial Search Fundamentals

Understanding competitive environments where agents work against each other

02

Why Games Matter

Exploring games as perfect testbeds for AI algorithms and reasoning

03

Problem Setup & Formulation

Learning how to model games mathematically for AI systems

04

Tic-Tac-Toe Example

Walking through a concrete example to understand game trees

05

Minimax Algorithm

The fundamental strategy for optimal play in two-player games

06

Alpha-Beta Pruning

Making minimax efficient by eliminating unnecessary calculations

What is Adversarial Search? 🤔

Adversarial search represents a major shift from the search problems we studied in previous chapters. Instead of working alone toward a goal, we now face intelligent opponents actively working against us!

🎯 Two-Player Games

Exactly two agents competing against each other, taking turns to make moves

👀 Fully Observable

Both players can see the complete game state - no hidden information or uncertainty

⚡ Deterministic

Actions have predictable outcomes - no random elements or chance involved

🏆 Zero-Sum Competition

One player's gain equals the other's loss - pure competition with clear winners

> "In which we examine the problems that arise when we try to plan ahead in a world where other agents are planning against us."

Why Study Games? 🎲

Games provide an ideal laboratory for developing and testing AI algorithms. They combine the complexity of real-world decision-making with the structure needed for computational analysis.

Perfect Mix of Logic & Creativity

Games require both systematic reasoning and innovative thinking - the hallmarks of human intelligence. They challenge AI systems to balance calculation with intuition.

Well-Defined Structure

Unlike messy real-world problems, games have clear rules, objectives, and win conditions. This structure makes them computationally tractable while still being intellectually challenging.

Fascinating Complexity

Despite simple rules, games can be incredibly complex. Chess has roughly 10^154 possible games, while Go has around 10^661 - numbers that dwarf the atoms in the observable universe!

Fun Fact: The branching factor (b) represents average moves per position, while depth (d) is the game length. Chess: b=35, d=100. Go: b=441, d=250!

Game Setup: MAX vs MIN 🥊

👑 The MAX Player

- Always moves first in our convention
- Tries to **maximize** the game's utility value
- Represents "us" - the AI we're designing
- Seeks the highest possible score

👹 The MIN Player

- Responds to MAX's moves
- Tries to **minimize** the utility value
- Represents our opponent
- Works to give us the lowest score possible

Turn-Taking Protocol

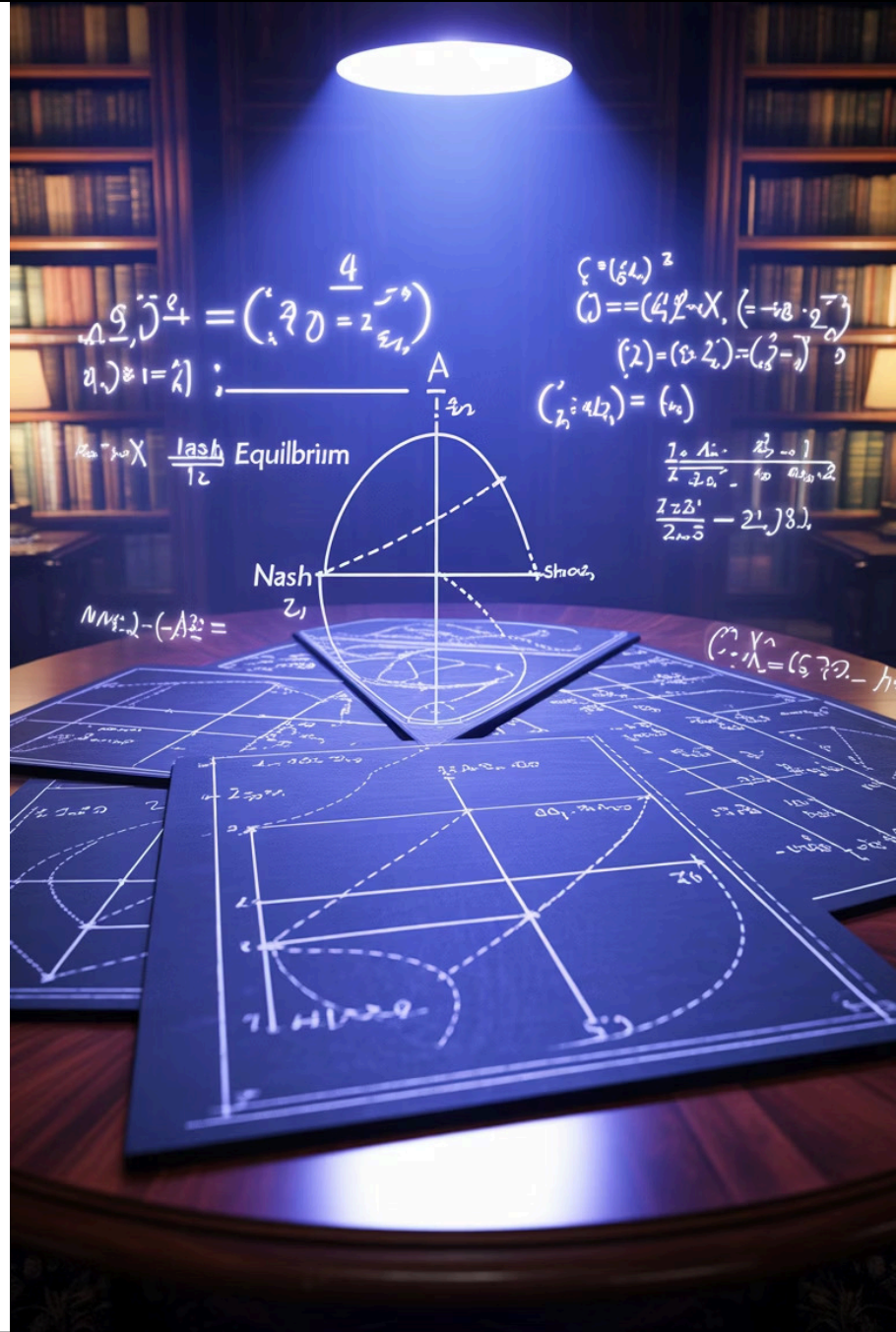
Players alternate moves until the game reaches a terminal state - either someone wins or it's a draw

Zero-Sum Nature

Total payoffs always sum to zero. In tic-tac-toe: Win(+1), Loss(-1), Draw(0). One player's success is exactly the other's failure

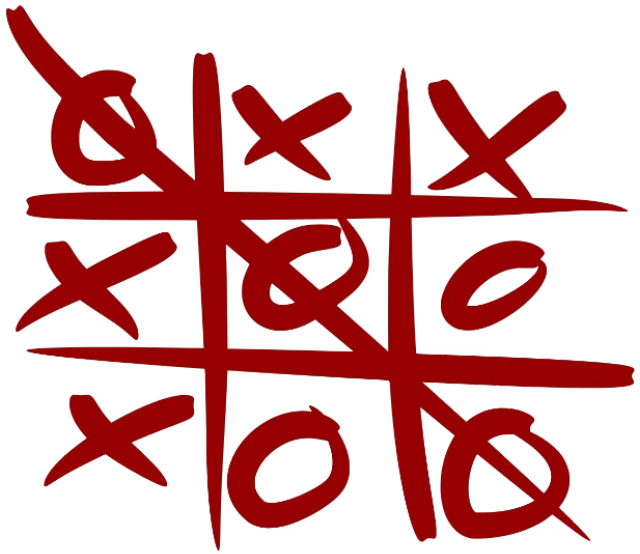


Problem Formulation



To solve games algorithmically, we need to formalize them mathematically. This involves defining states, actions, transitions, and utility functions precisely.

Example: Tic-Tac-Toe 🎯

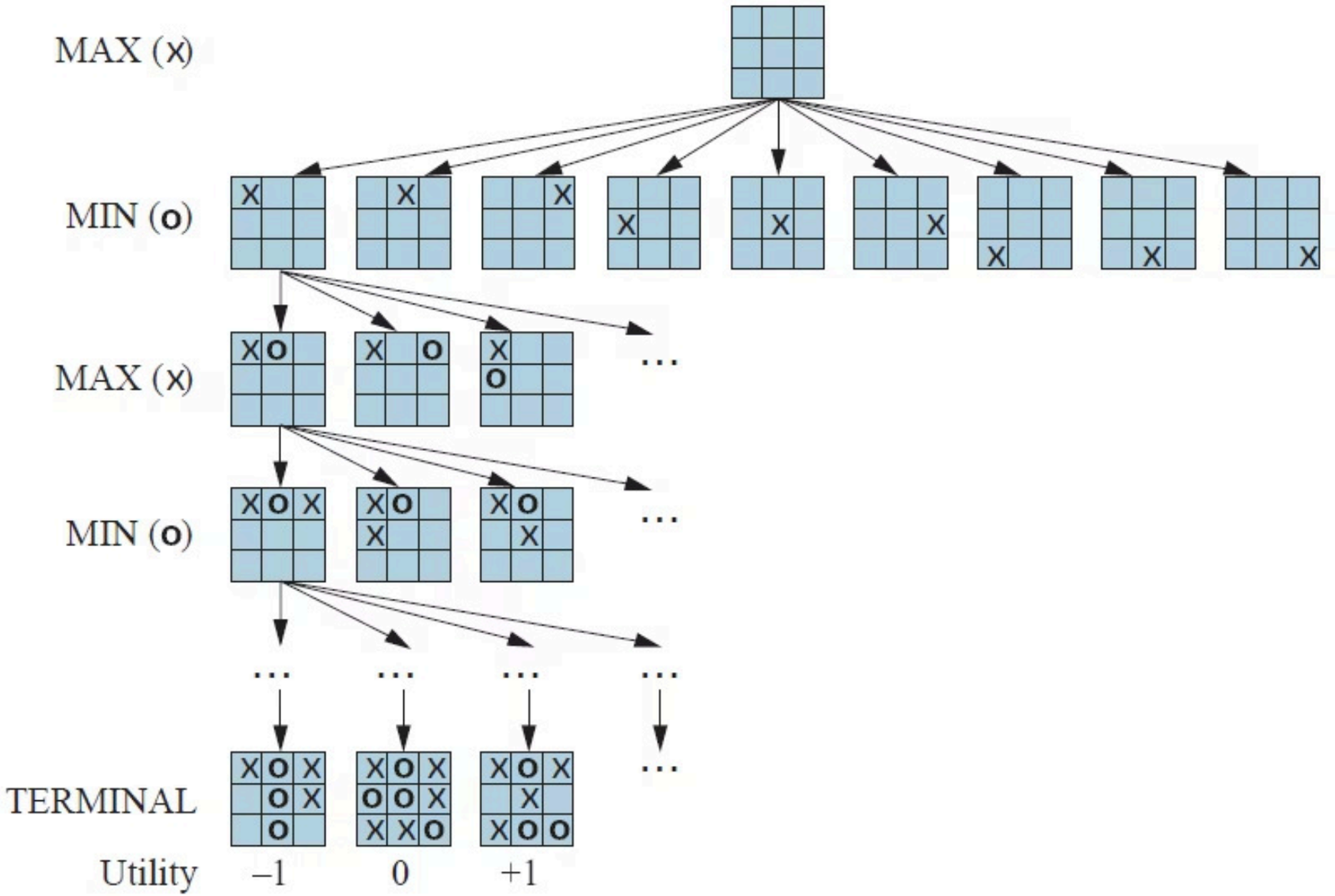


Game Components 📝

- **Initial State:** Empty 3×3 grid
- **Players:** X (MAX) and O (MIN)
- **Actions:** Place symbol in empty square
- **Terminal Test:** Three in a row or board full
- **Utility:** +1 (X wins), -1 (O wins), 0 (draw)

Here, player X represents MAX (trying to maximize utility) while player O represents MIN (trying to minimize utility). This simple game perfectly illustrates adversarial search principles!

Tic-Tac-Toe: Game Tree Structure 🌳



Tic-Tac-Toe: Game Tree Structure 🌳

🎯 States/Nodes

Each node represents a board configuration. The root is the empty board, leaves are terminal states (someone won or drew)

➡️ Actions/Edges

Edges represent legal moves - placing X or O in an available square. Each action transitions from one state to another

🔢 Node Count

From the initial state, X has 9 possible first moves. The tree grows exponentially, though many paths lead to equivalent positions

🏁 Terminal Utilities

Leaf nodes have utility values: +1 if X wins, -1 if O wins, 0 for draws. These values propagate up the tree

The game tree captures all possible ways the game could unfold. While tic-tac-toe's tree is manageable, games like chess have astronomically large trees!

Methods for Adversarial Games 🛠️

🎯 Exact Methods

Perfect Play Algorithms

Minimax Search: Assumes both players play optimally. Explores the entire game tree to find the truly best move.

Alpha-Beta Pruning: Optimizes minimax by eliminating branches that can't affect the final decision, dramatically reducing computation.

Non-deterministic Modeling: Treats the opponent as an unpredictable environment, considering all possible opponent responses.

🎲 Heuristic Methods

Practical Approximations

Heuristic Alpha-Beta: Uses evaluation functions to estimate positions without reaching terminal states. Essential for complex games.

Forward Pruning: Ignores clearly bad moves early to focus computational resources on promising alternatives.

Monte Carlo Tree Search: Uses random game simulations to estimate position values - the secret behind modern Go AI!

Today we'll focus on exact methods - the mathematical foundations that guarantee optimal play in games where perfect analysis is computationally feasible.



Our Focus: Exact Methods 🔍

1

Minimax Algorithm

The fundamental approach for finding optimal moves in two-player zero-sum games. Assumes both players make perfect decisions.

2

Alpha-Beta Pruning

An optimization that produces identical results to minimax but with dramatically improved efficiency through intelligent branch elimination.

These exact methods guarantee optimal play - they find moves that are mathematically proven to be the best possible, assuming the opponent also plays perfectly. While computationally intensive, they provide the theoretical foundation for all game-playing AI! 🧠✨



The Minimax Idea 💡

The core insight of minimax is elegantly simple: **maximize your minimum guaranteed outcome**. By assuming your opponent plays perfectly against you, you can choose moves that give you the best result in the worst-case scenario. This pessimistic but rational approach ensures optimal play! 🤔

Minimax Example: Terminal Values 🎯

Let's walk through minimax step by step! We start at the terminal nodes (game endings) where we can directly calculate utility values from our utility function.

1 Identify Terminal States

Find all leaf nodes in the game tree - positions where the game has ended (win/loss/draw)

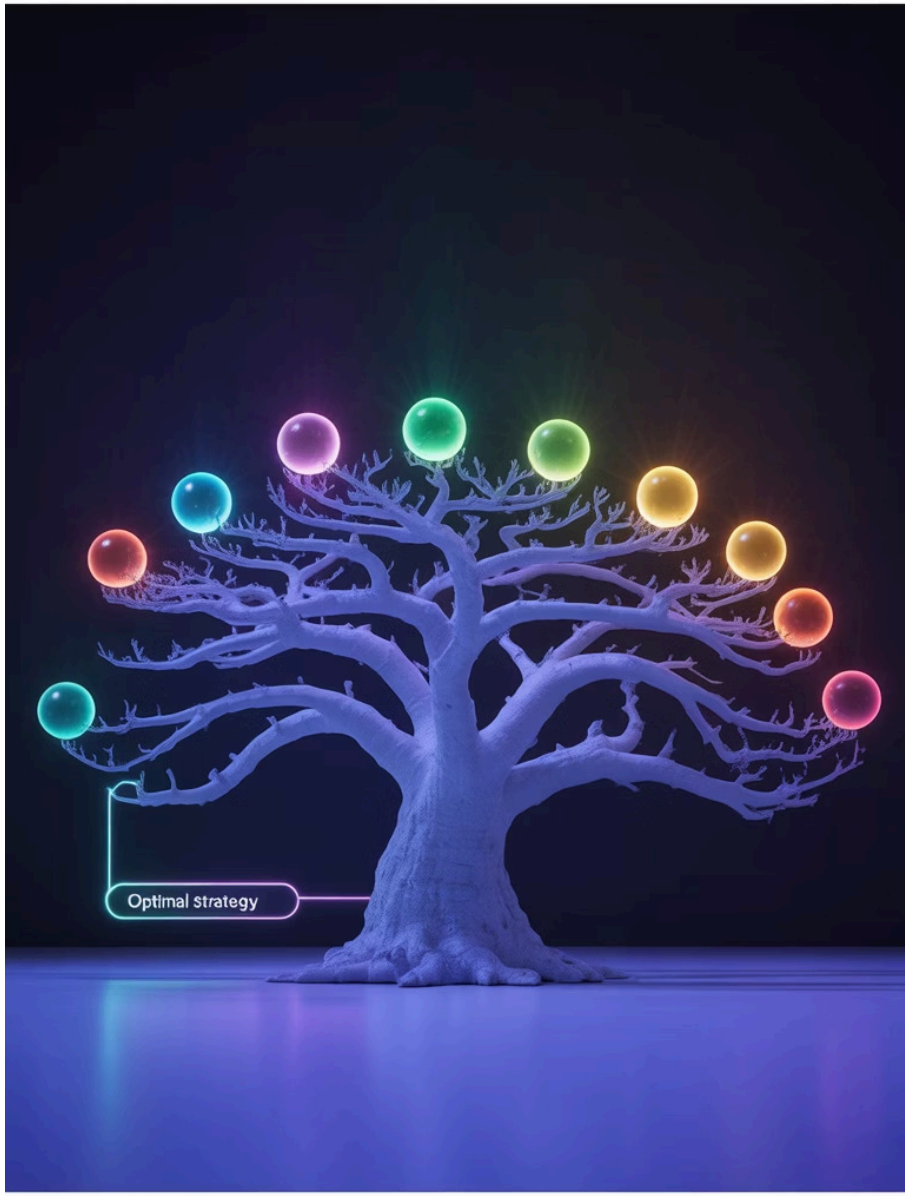
2 Apply Utility Function

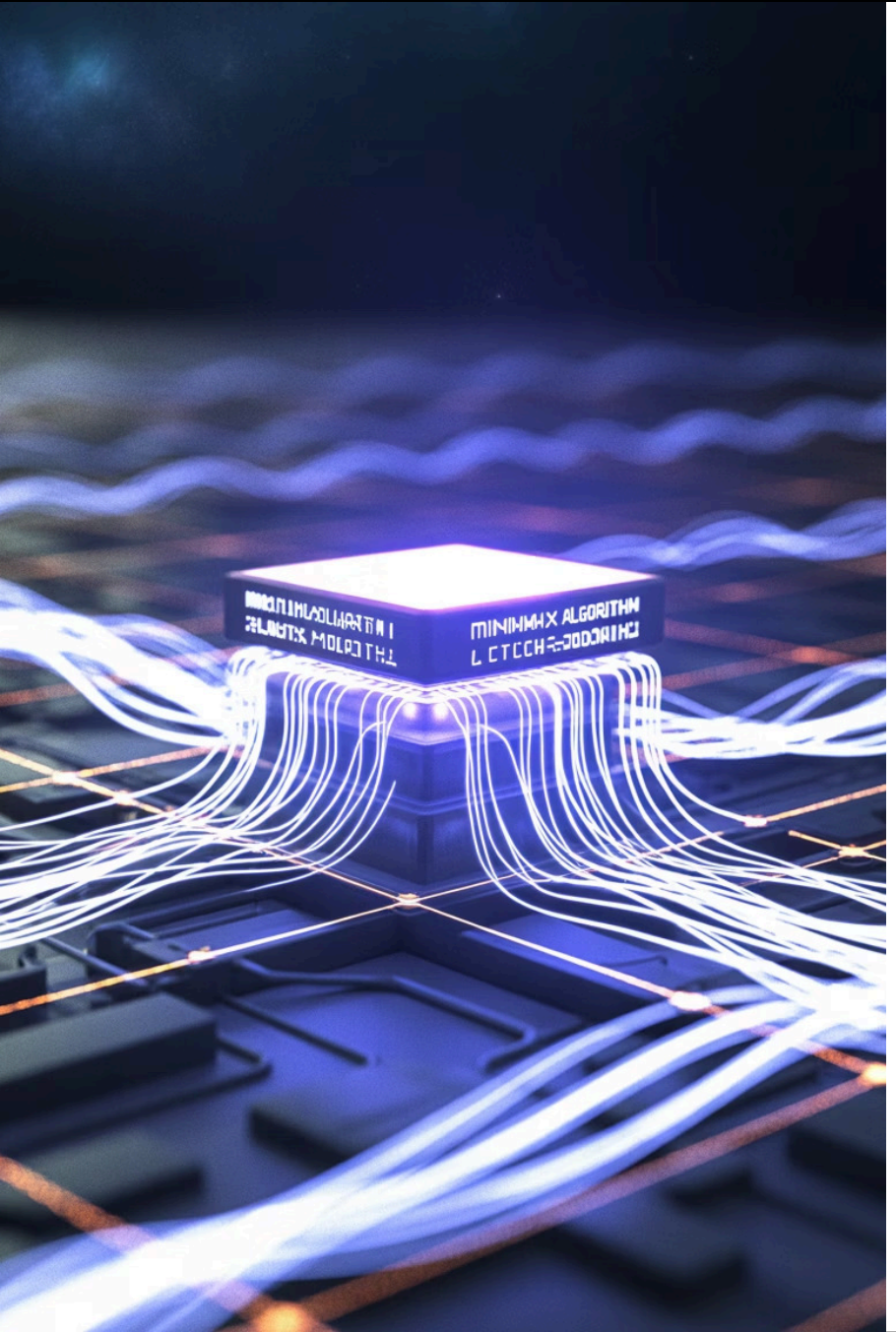
Calculate the exact utility value for each terminal state using our predefined scoring system

3 Mark the Values

Label each terminal node with its utility value - these become the foundation for the entire algorithm

Remember: Terminal values are the only "real" scores in the tree. All other values are computed by the minimax algorithm based on optimal play assumptions! 🎲





Minimax Example: Internal Node Values 🧮

Now the magic happens! We propagate values up the tree using the minimax rule: MAX nodes take the maximum of their children, MIN nodes take the minimum.

Bottom-Up Calculation

Start from terminal nodes and work upward, level by level, until reaching the root

MAX Node Rule

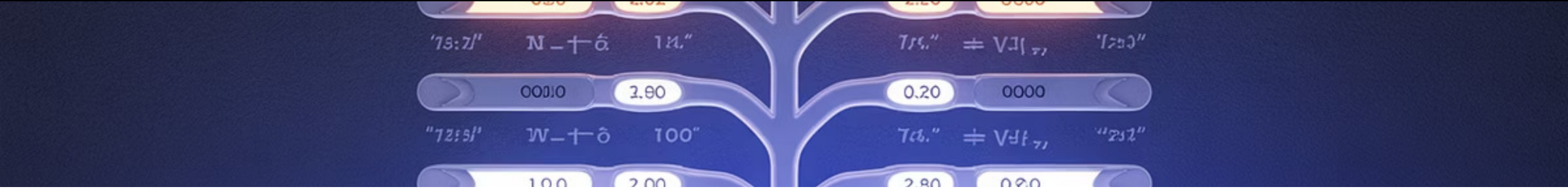
MAX nodes (our moves) choose the child with the highest utility value

MIN Node Rule

MIN nodes (opponent moves) choose the child with the lowest utility value

Optimal Strategy Emerges

This process reveals the best move sequence assuming perfect play by both sides



Minimax Example: Step-by-Step 📊

Watch how values bubble up through the tree! Each internal node's value depends on whether it's a MAX or MIN node:

- **MIN nodes** (○) choose the *smallest* child value
- **MAX nodes** (□) choose the *largest* child value
- This continues until we reach the root

The root's final value tells us the outcome of perfect play, and we can trace back to find the optimal first move!

01

Evaluate Terminals

Calculate leaf utilities

02

MIN Layer

Each MIN node takes minimum child

03

MAX Layer

Each MAX node takes maximum child

04

Continue Upward

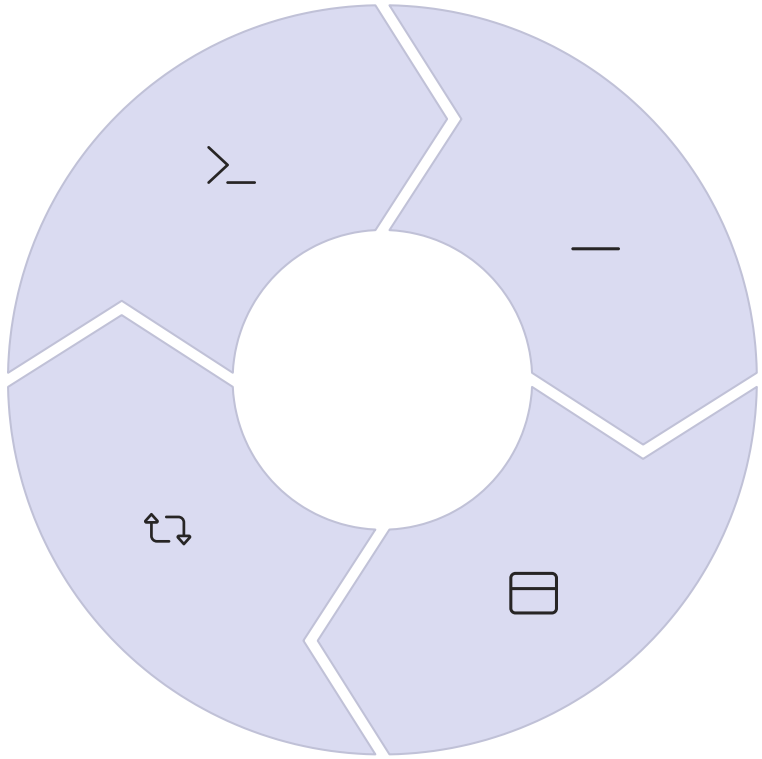
Repeat until reaching root

Minimax Example: More Propagation ⬆️

As we continue propagating values upward, the pattern becomes clear. Each node's value represents the best outcome achievable from that position, assuming both players play optimally from that point forward.

Terminal Values

Direct utility calculations from game outcomes



MIN Chooses

Opponent selects move that minimizes our utility

Values Propagate

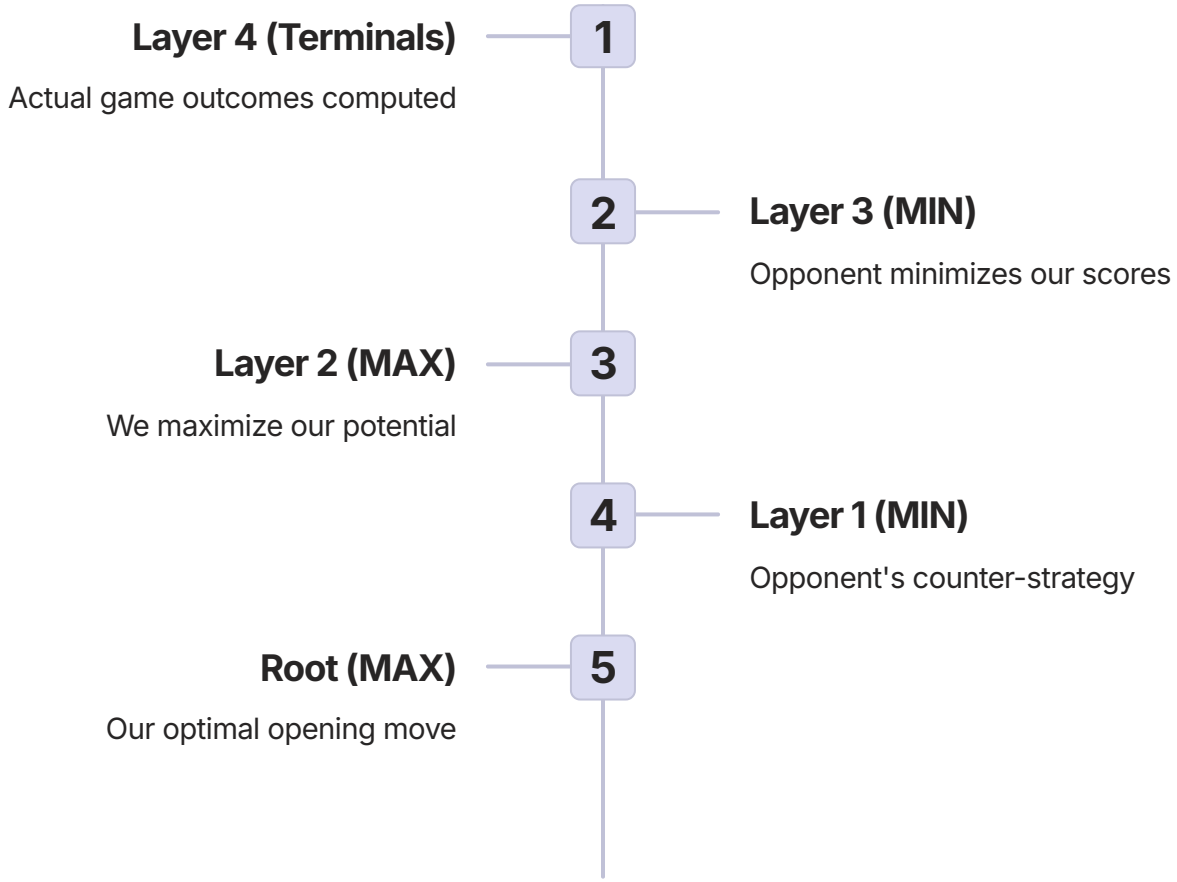
Process continues up the entire tree

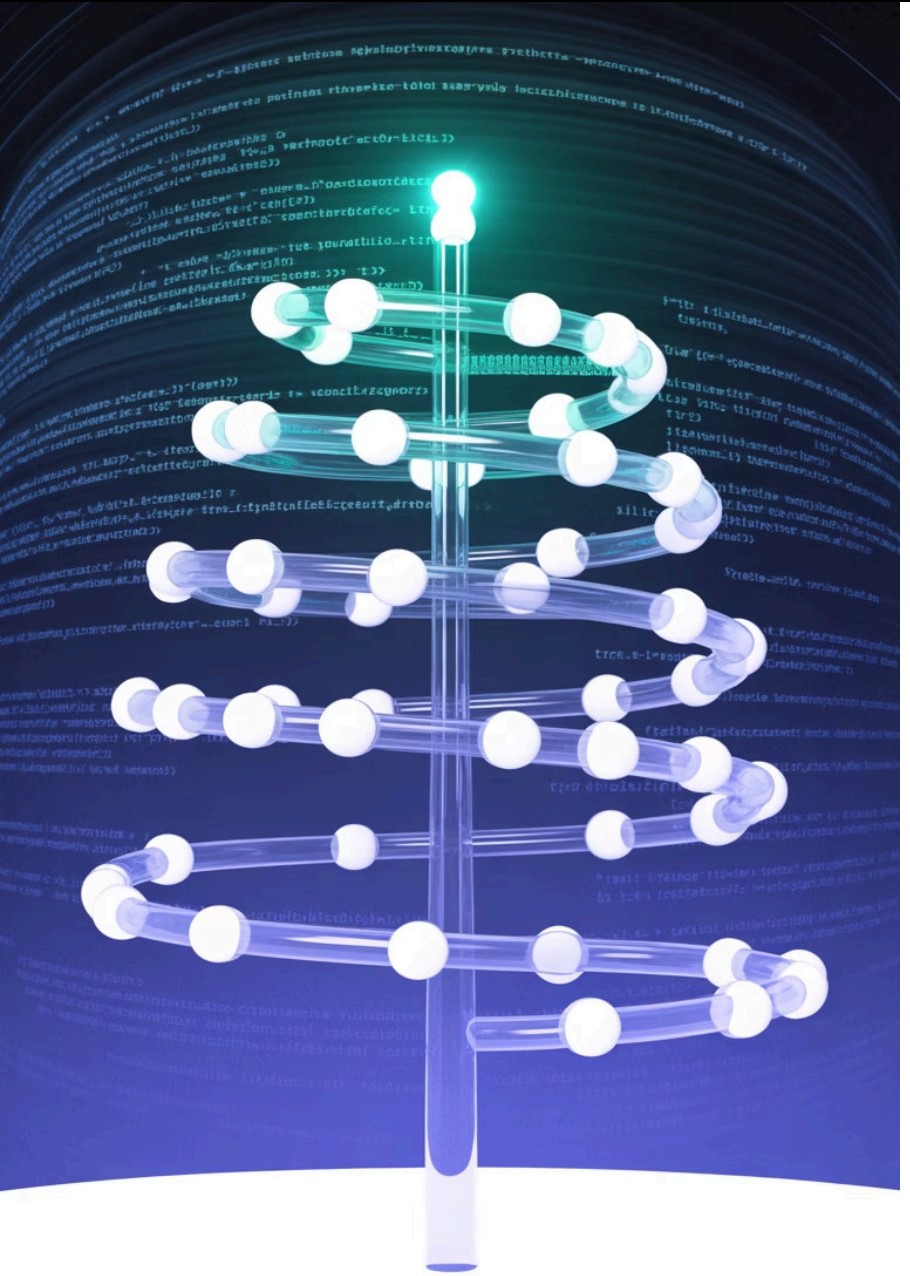
MAX Chooses

We select move that maximizes our utility

Minimax Example: Near Completion 🔄

We're almost there! The values are propagating up toward the root node. Notice how each level alternates between MAX and MIN decisions, creating a back-and-forth negotiation between the players' optimal strategies.





Minimax Example: Final Result! 🏆

> **Victory!** 🎉 The minimax algorithm has found the optimal strategy. The root node shows our best guaranteed outcome, and we can trace the path to find our optimal first move!

Optimal Move Identified

The child of the root with the highest value reveals our best opening move

Complete Strategy

We can trace optimal responses for any opponent move by following the tree

Mathematical Guarantee

This move guarantees the best possible outcome against perfect play

The beauty of minimax is its guarantee: no matter how brilliantly the opponent plays, we cannot do better than this move! 🧠✨

Minimax Algorithm Pseudocode 💻

1

MINIMAX-DECISION

```
function MINIMAX-DECISION(state)
returns an action
  inputs: state, current state in game
  return arg max[a in Actions(state)]
Min-Value(Result(state,a))
```

Main function that finds the action leading to maximum utility 🎯

2

MAX-VALUE

```
function MAX-VALUE(state) returns a
utility value
  if TERMINAL-TEST(state) then return
UTILITY(state)
  v ← -∞
  for a in ACTIONS(state) do
    v ← MAX(v, MIN-
VALUE(Result(state,a)))
  return v
```

Maximizing player seeks highest possible utility 📈

3

MIN-VALUE

```
function MIN-VALUE(state) returns a
utility value
  if TERMINAL-TEST(state) then return
UTILITY(state)
  v ← +∞
  for a in ACTIONS(state) do
    v ← MIN(v, MAX-
VALUE(Result(state,a)))
  return v
```

Minimizing player seeks lowest possible utility 📉

The Computational Challenge 😰

35

Chess Branching

Average moves per position

100

Chess Depth

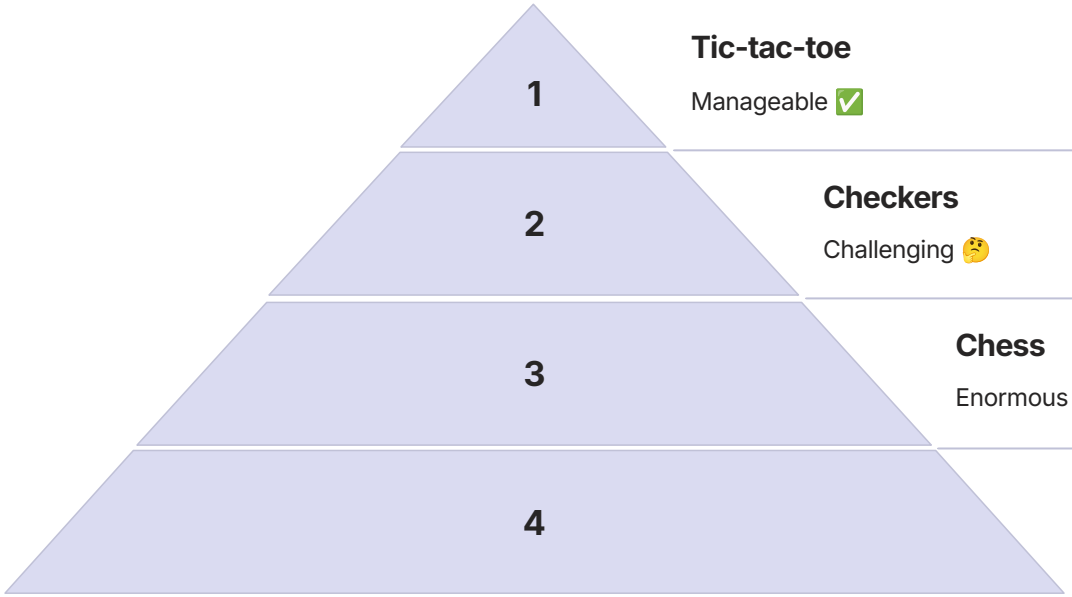
Typical game length

10154

Total States

Possible chess games (roughly!)

The game tree size grows as **O(b^d)** where b is branching factor and d is maximum depth. For chess, this means roughly 35^100 nodes - more than atoms in the observable universe! 🌌



Tic-tac-toe

Manageable ✅

Checkers

Challenging 🤔

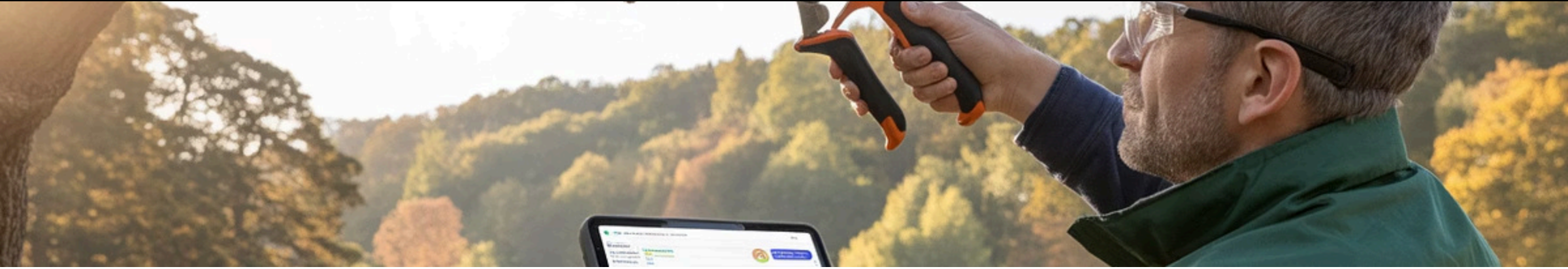
Chess

Enormous 😅

Go

Astronomical! 🤯

⚠ Pure minimax quickly becomes computationally infeasible. We need smarter approaches! Enter alpha-beta pruning... 🚀



Alpha-Beta Pruning ✂️

The brilliant insight: we don't need to examine every branch! If we can prove that a branch won't affect the final decision, we can safely ignore it. This can reduce the search space from exponential to nearly manageable! 🎯✨

Alpha-Beta Example: Setup 🏗️

Let's watch alpha-beta pruning in action! We maintain two values as we search:

α (Alpha) - MAX's Best 👑

Lower bound: The best value MAX can guarantee so far. Starts at -∞ and only increases.

β (Beta) - MIN's Best 👹

Upper bound: The best value MIN can guarantee so far. Starts at +∞ and only decreases.

Pruning Rule ✂️

If α ≥ β, prune! This means MAX's best option is already better than MIN's best option in this branch

Key Insight 💡

We can ignore branches that provably won't influence the final decision

ⓘ Remember: Terminal node values are discovered only when we actually visit them! 🎯



Alpha-Beta Step 1: Initialize 🚀

We begin our alpha-beta search at the root with initial values:

- **α = -∞** (MAX's best so far)
- **β = +∞** (MIN's best so far)

As we explore the tree depth-first, we'll update these values and watch for pruning opportunities. The moment α ≥ β, we can stop exploring that branch! ✂️

Notice how we label each node with its current α and β values. This helps us track the bounds and identify when pruning becomes possible.

🎯 Goal

Find the same optimal move as minimax, but faster!

📊 Strategy

Maintain bounds and prune impossible branches



Alpha-Beta Step 2: First Terminal 🎯

We've reached our first terminal node with value 10! Now watch how this affects our α and β values as we backtrack up the tree.

Terminal Found

First leaf node evaluated: utility = 10

MIN Node Updates

Parent MIN node: β changes from +∞ to 10

Values Propagate

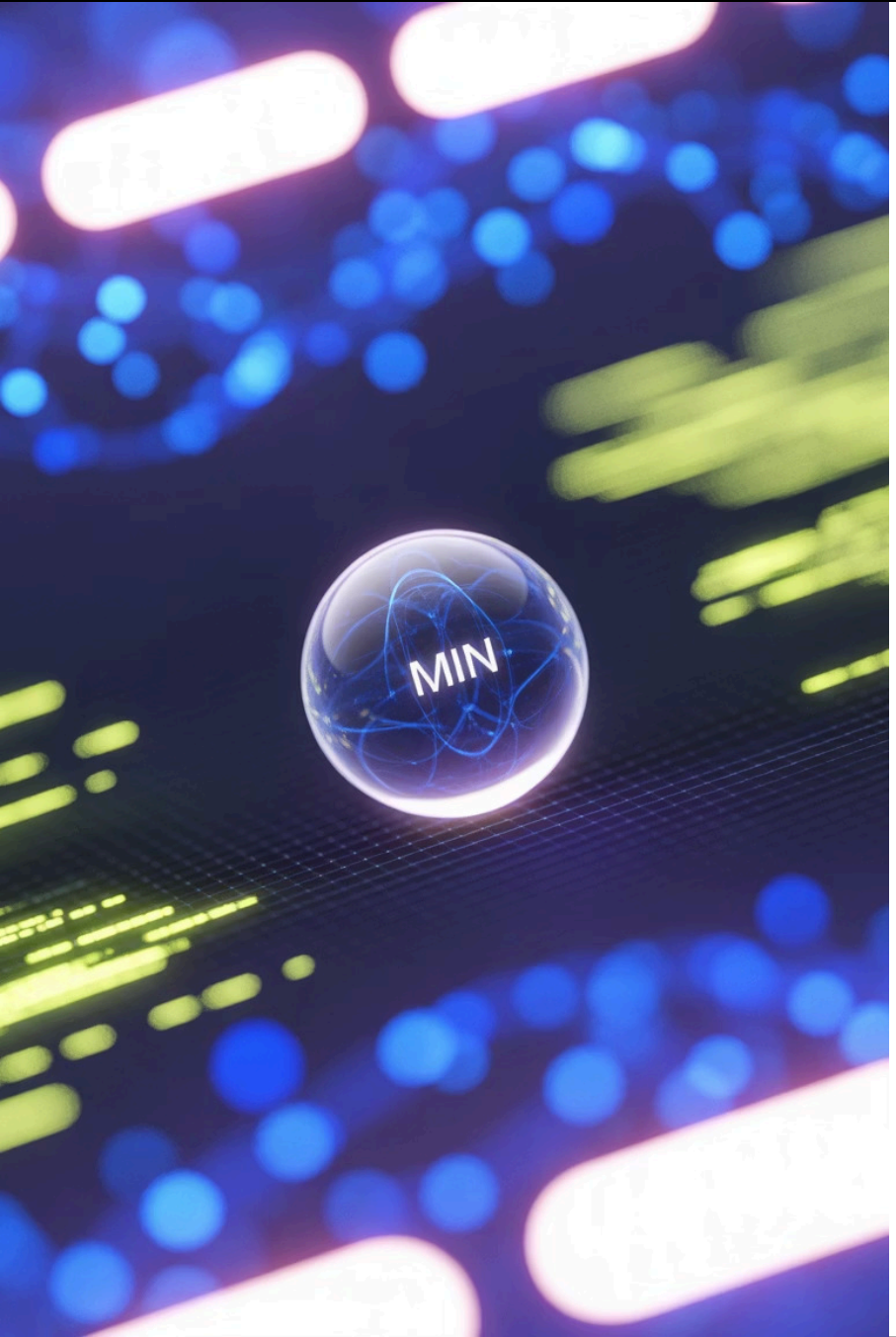
Information flows back up toward root

Check Pruning

α = -∞, β = 10. Since α < β, continue searching

> 🔍 **Key insight:** We update β because this is a MIN node - it wants to minimize the value, so it keeps track of the smallest value seen so far.





Alpha-Beta Step 3: MIN Node Completes 📉

The MIN node has evaluated both its children (10 and 9) and chooses the minimum value of 9. This becomes the node's final value, which then propagates to its MAX parent.

MIN Chooses Minimum

Between values 10 and 9, MIN selects 9 (the smaller value)

Value Propagates Up

The value 9 becomes available to the parent MAX node

Alpha Updates

Parent MAX node updates α from -∞ to 9 (its best option so far)

Progress! 🎉 We now know MAX can guarantee at least a utility of 9. Any path that offers less can potentially be pruned.

Alpha-Beta Step 4: Second Branch 🌿

Now we explore the second major branch. The α value of 10 propagates down, representing MAX's current best guarantee. Watch what happens when we find a value that's worse than this guarantee!

Current Situation 📊

- MAX has α = 10 (can guarantee utility of 10)
- We're exploring a new MIN subtree
- First terminal in this branch has value 5
- MIN node updates β from +∞ to 5

Pruning Check ✂️

- Current values: α = 10, β = 5
- Since α ≥ β (10 ≥ 5), we can prune!
- No need to check remaining children
- This MIN branch can't improve MAX's position

> **Eureka!** 💡 Our first pruning opportunity has arrived. Since MAX already has a guaranteed utility of 10, and this MIN branch can only offer 5 or less, we can safely ignore the rest of this subtree!



Alpha-Beta Step 5: More Exploration 🔍

We continue our systematic exploration of the game tree, carrying our updated α and β values to new branches. Each completed subtree gives us more information for making pruning decisions.

01

Evaluate New Subtree

Move to next unexamined branch with current bounds

02

Update Bounds

Modify α and β based on node type and values found

03

Check for Pruning

If α ≥ β, eliminate remaining children in current branch

04

Backtrack Results

Propagate final values up to parent nodes

The systematic nature of alpha-beta ensures we never miss important branches while eliminating those that can't affect the final decision. It's the same result as minimax, just much faster! ⚡

Alpha-Beta Step 6: Pattern Recognition 🎨

A beautiful pattern emerges! Notice how alpha-beta intelligently skips entire subtrees (shown with dashed lines) that cannot possibly influence the final decision. This is the algorithm's power in action! ✨

Strategic Elimination

Branches are pruned based on mathematical certainty, not guesswork. We know these paths cannot lead to better decisions.

Efficiency Gains

Each pruned subtree saves exponential amounts of computation. Small prunings early have massive downstream effects.

Same Optimal Result

Despite examining fewer nodes, alpha-beta produces identical results to full minimax search.

Mathematical Beauty! 📐 Alpha-beta proves that logical reasoning can dramatically outperform brute-force computation.



Alpha-Beta Step 7: Root Value Found 🏁

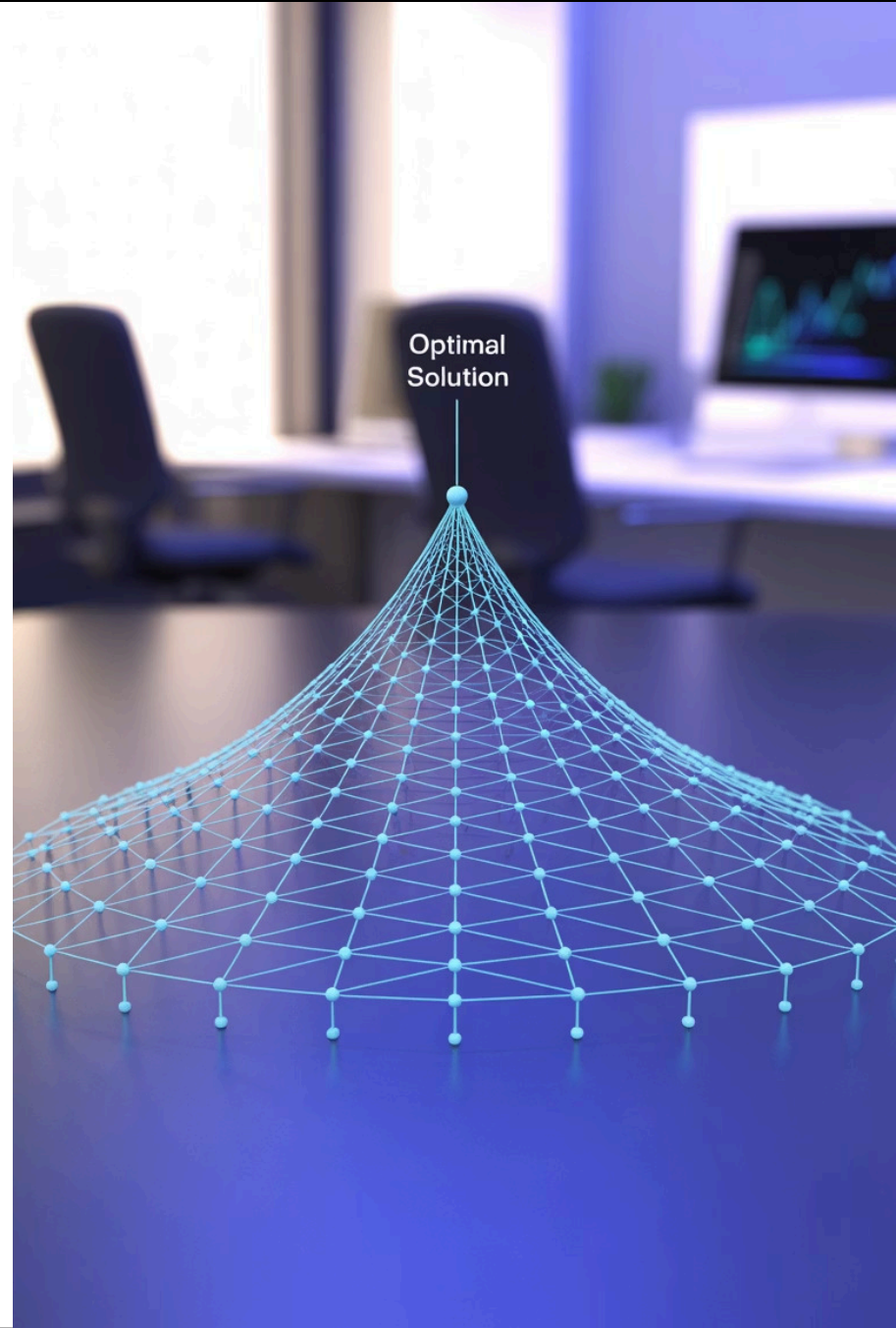
Success! The alpha-beta algorithm has determined the root's value, identifying the optimal move for MAX. We achieved the same result as pure minimax but examined far fewer nodes! 🎊

10	40%	100%
Optimal Value	**Nodes Pruned**	**Accuracy**
Best outcome for MAX	Approximate savings	Same as full minimax

The root node's final value of 10 tells us:

- MAX can guarantee a utility of 10 with perfect play
- The optimal first move leads to this outcome
- No better strategy exists for MAX
- MIN's best counter-strategy is accounted for

Mission Accomplished! 🚀
Alpha-beta has found the provably optimal move while saving substantial computation.



Alpha-Beta: More Examples 📚

Let's see more alpha-beta scenarios to fully understand when and how pruning occurs. Each example demonstrates different pruning patterns and their computational savings.

1. **Early Pruning**
   Best case: prune after examining just one child of MIN nodes
2. **Late Pruning**
   Worst case: prune only after examining most children
3. **No Pruning**
   Rare case: examination order prevents any pruning opportunities
4. **Optimal Ordering**
   Best moves first maximizes pruning potential

Pro Tip! 💡 The order of move examination significantly affects pruning efficiency. Examining better moves first leads to more pruning opportunities!



Alpha-Beta: Continuing the Example 🔄

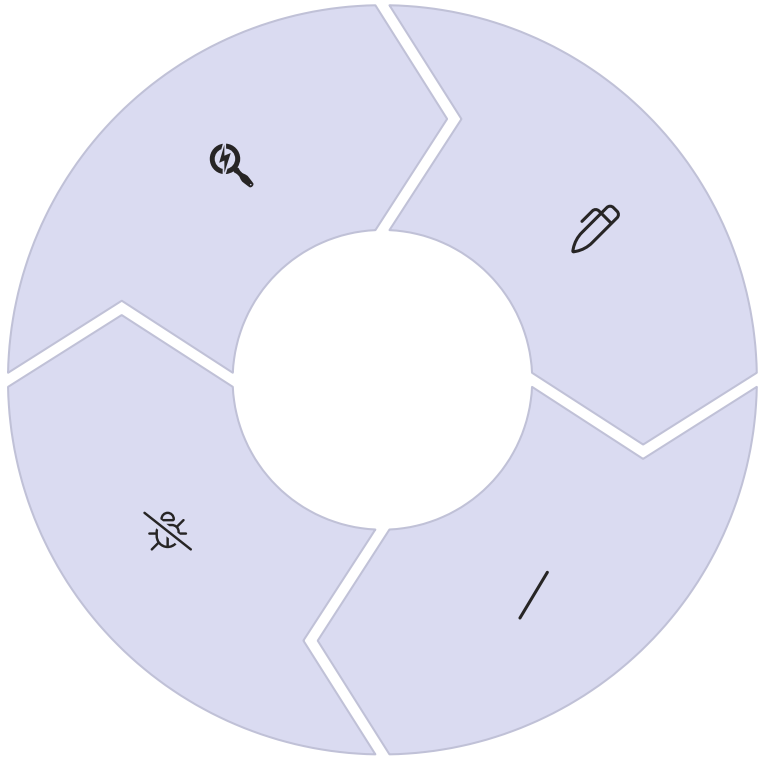
As our alpha-beta search continues, we see the interplay between bound updates and pruning decisions. Each completed subtree provides more information for eliminating unpromising branches.

Systematic Exploration

Depth-first search with bound propagation

Bound Updates

Alpha and beta values continuously refined



Efficiency Gains

Exponential speedup through intelligent search

Pruning Decisions

Mathematical certainty guides branch elimination

The beauty of alpha-beta lies in its guarantee: we get the exact same optimal move as minimax, but often in a fraction of the time! ⚡



Alpha-Beta: Near the End 🎬

We're approaching the completion of our alpha-beta search! Notice how many branches have been eliminated (shown with dashed lines). Each pruned subtree represents significant computational savings without any loss in decision quality.

🎯 Branches Explored

Only the necessary paths for optimal decision-making

✂️ Branches Pruned

Mathematically proven to be irrelevant

⚡ Time Saved

Exponential reduction in computational complexity

🏆 Result Quality

Identical to exhaustive minimax search

> The pruned branches aren't "mistakes" - they're intelligent eliminations based on logical deduction! 🧠✨

Alpha-Beta: Final Pruning Opportunities ✂️

In our final steps, alpha-beta continues to find pruning opportunities. The algorithm maintains its vigilance throughout the entire search process, never missing a chance to eliminate unnecessary computation.

Continuous Vigilance

Algorithm checks for pruning at every node, maintaining efficiency throughout the search

Compound Savings

Early pruning decisions have cascading effects, eliminating entire subtrees of computation

Perfect Accuracy

Despite aggressive pruning, the final result is mathematically guaranteed to be optimal

Efficiency Masterclass! 🎓 Alpha-beta demonstrates how mathematical insight can transform brute-force algorithms into elegant, efficient solutions.



Alpha-Beta: Complete! 🏁

> 🎉 **Mission Accomplished!** Alpha-beta pruning has successfully identified the optimal move while examining significantly fewer nodes than pure minimax would require.

Optimal Decision Found

The root value reveals the best possible outcome and corresponding move

Massive Efficiency Gain

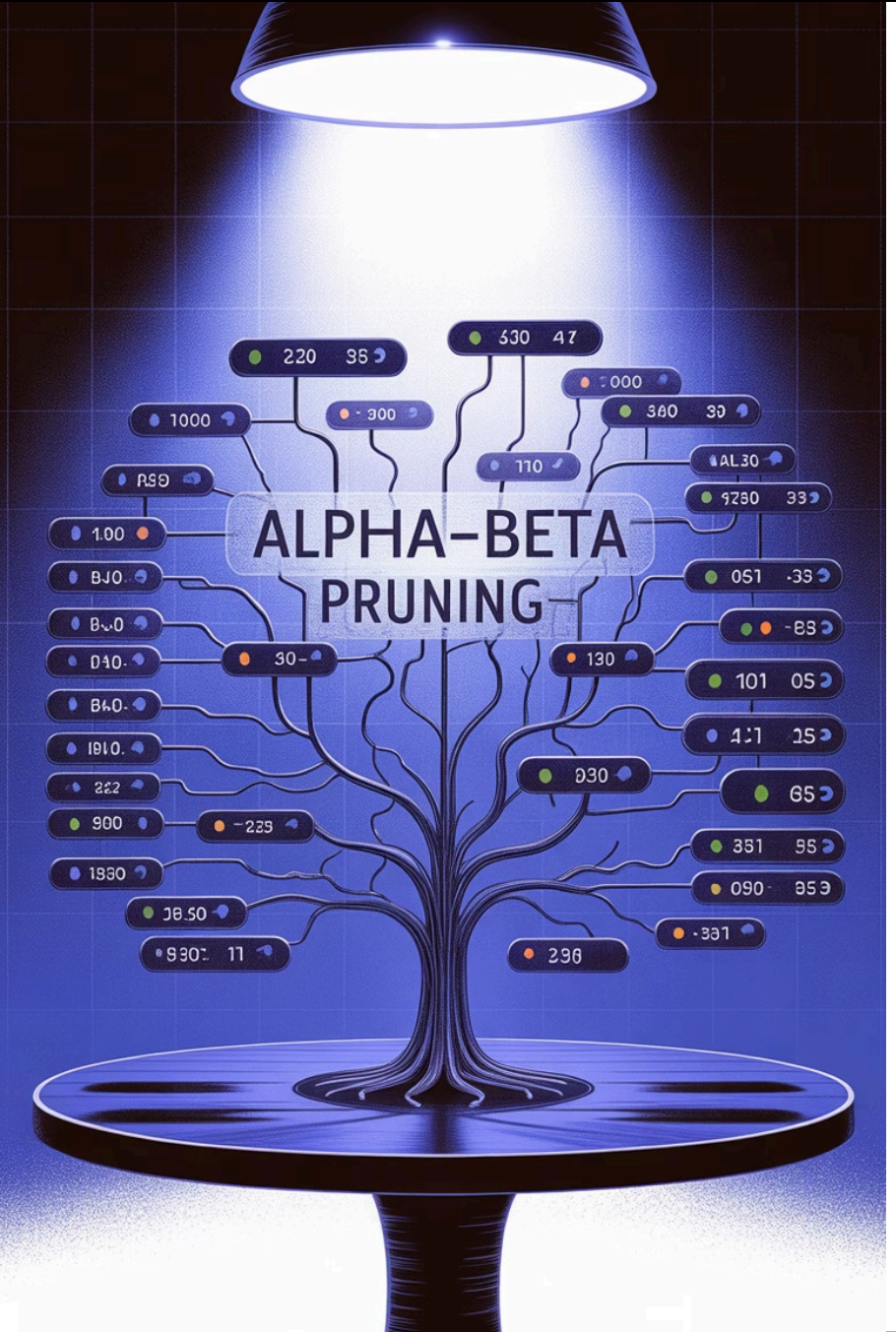
Eliminated numerous branches without compromising decision quality

Mathematical Certainty

Results are provably identical to exhaustive minimax search

This elegant algorithm demonstrates the power of combining logical reasoning with systematic search. It's the foundation upon which modern game-playing AI is built! 🤖✨





Alpha-Beta: Another Complete Example 📖

Let's examine another complete alpha-beta execution to solidify our understanding. This example shows different pruning patterns and reinforces the algorithm's consistent efficiency gains.

Key Observations 🔍

- Pruning locations depend on tree structure
- Node ordering affects pruning efficiency
- Early pruning saves the most computation
- Result is always identical to minimax

Learning Points 💡

- Alpha-beta is deterministic and reliable
- Best-case: examine O(b^(d/2)) nodes
- Worst-case: still O(b^d) like minimax
- Average case: significant improvement

Practice with different examples helps build intuition for recognizing pruning opportunities and understanding the algorithm's behavior in various scenarios! 🎯

Alpha-Beta: Final Example 🎭

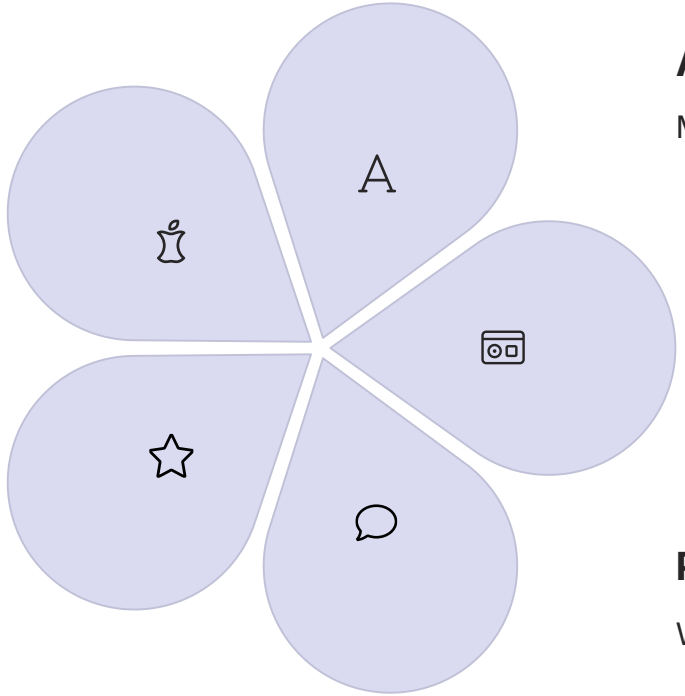
Our final alpha-beta example showcases the algorithm in its full glory. By now, you should be able to predict where pruning will occur and understand why certain branches can be safely eliminated! 🎓

Core Concept

Eliminate provably irrelevant branches

Efficiency

Same result, less computation



Alpha Bound

MAX's best guarantee

Beta Bound

MIN's best guarantee

Pruning Rule

When α ≥ β, eliminate subtree

Mastery Check! ✅ Can you identify the pruning opportunities in this tree before the algorithm finds them?

Alpha-Beta Algorithm Pseudocode 💻

1

ALPHA-BETA-SEARCH

```
function ALPHA-BETA-SEARCH(state)
returns an action
  v ← MAX-VALUE(state, -∞, +∞)
  return the action in ACTIONS(state) with
value v
```

Main entry point that initiates search with unbounded α and β 🚀

2

MAX-VALUE Function

```
function MAX-VALUE(state, α, β) returns a
utility value
  if TERMINAL-TEST(state) then return
UTILITY(state)
  v ← -∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    α ← MAX(α, v)
    if α ≥ β then return v    // β cut-off
  return v
```

Maximizing player with pruning logic 📈

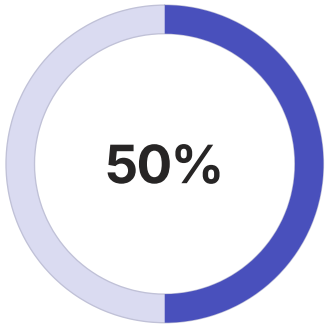
3

MIN-VALUE Function

```
function MIN-VALUE(state, α, β) returns a
utility value
  if TERMINAL-TEST(state) then return
UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a), α, β))
    β ← MIN(β, v)
    if α ≥ β then return v    // α cut-off
  return v
```

Minimizing player with pruning logic 📉

Alpha-Beta Performance Properties 📊



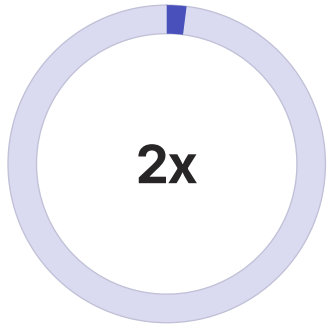
Best-Case Reduction

Examines only O(b^(d/2)) nodes instead of O(b^d)



Accuracy Maintained

Always produces identical results to pure minimax



Effective Depth

Can search twice as deep in the same time

🎯 Best Case

Perfect move ordering: When we examine the best moves first, pruning is maximized. The search tree effectively becomes O(b^(d/2)), allowing us to search twice as deep!

😐 Worst Case

Poor move ordering: When the best moves are examined last, minimal pruning occurs. Performance degrades to O(b^d), same as minimax.

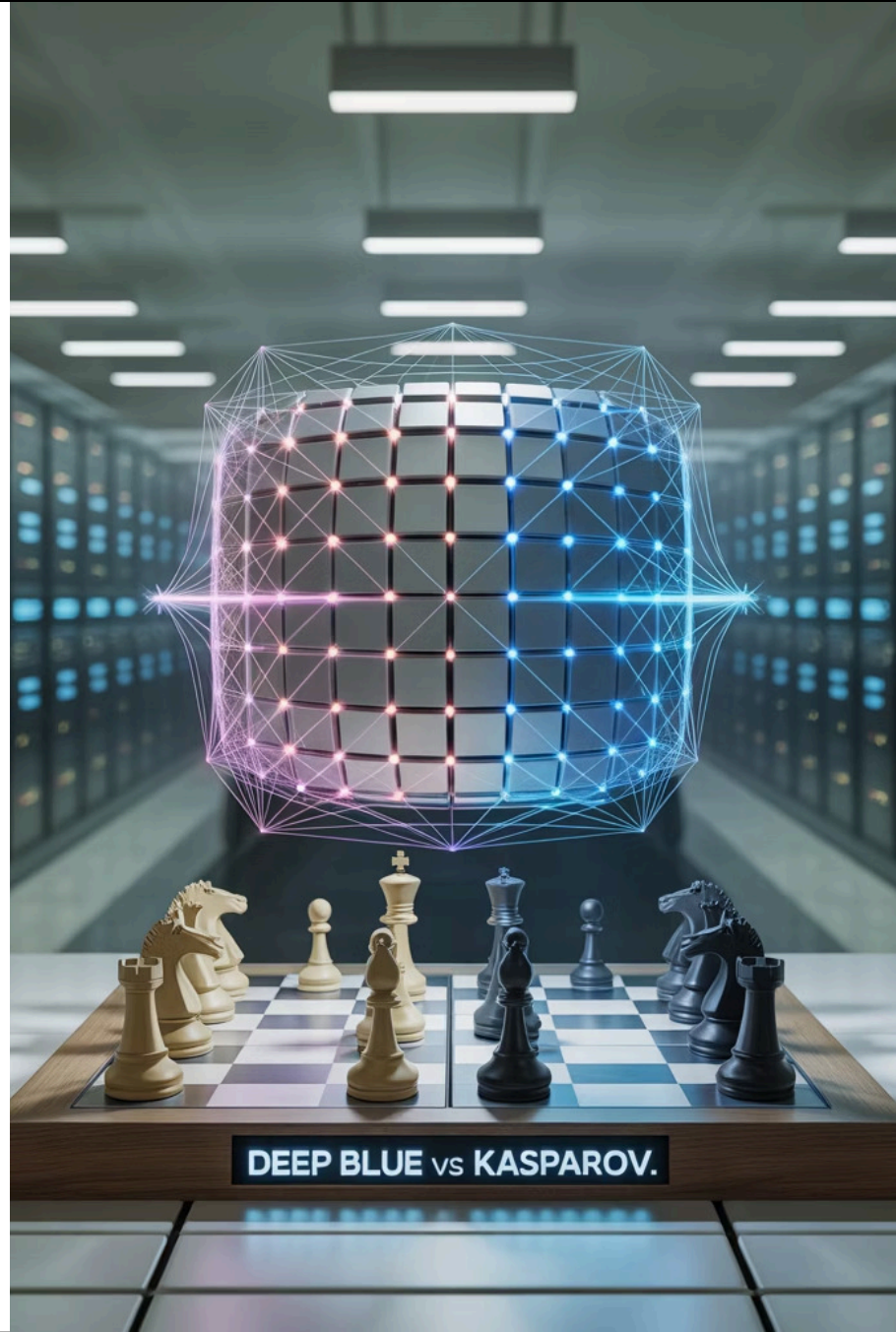
📈 Average Case

Random move ordering: Typically achieves O(b^(3d/4)), providing substantial improvement over pure minimax in most practical situations.

♟️ Applications: What Algorithm Did Deep Blue Use?

- 🔑 **Core Algorithm:** Minimax + Alpha-Beta Pruning
  - Standard adversarial search
  - Explored move sequences for both players
  - Alpha-Beta allowed **deeper lookahead**
- ⚡ **Enhancements:**
  - **Massive parallel search** (200M positions/sec, 6–20 ply)
  - **Sophisticated evaluation function** (material, king safety, mobility, pawn structure)
  - **Opening book** (professional opening lines)
  - **Endgame databases** (perfect play with few pieces)

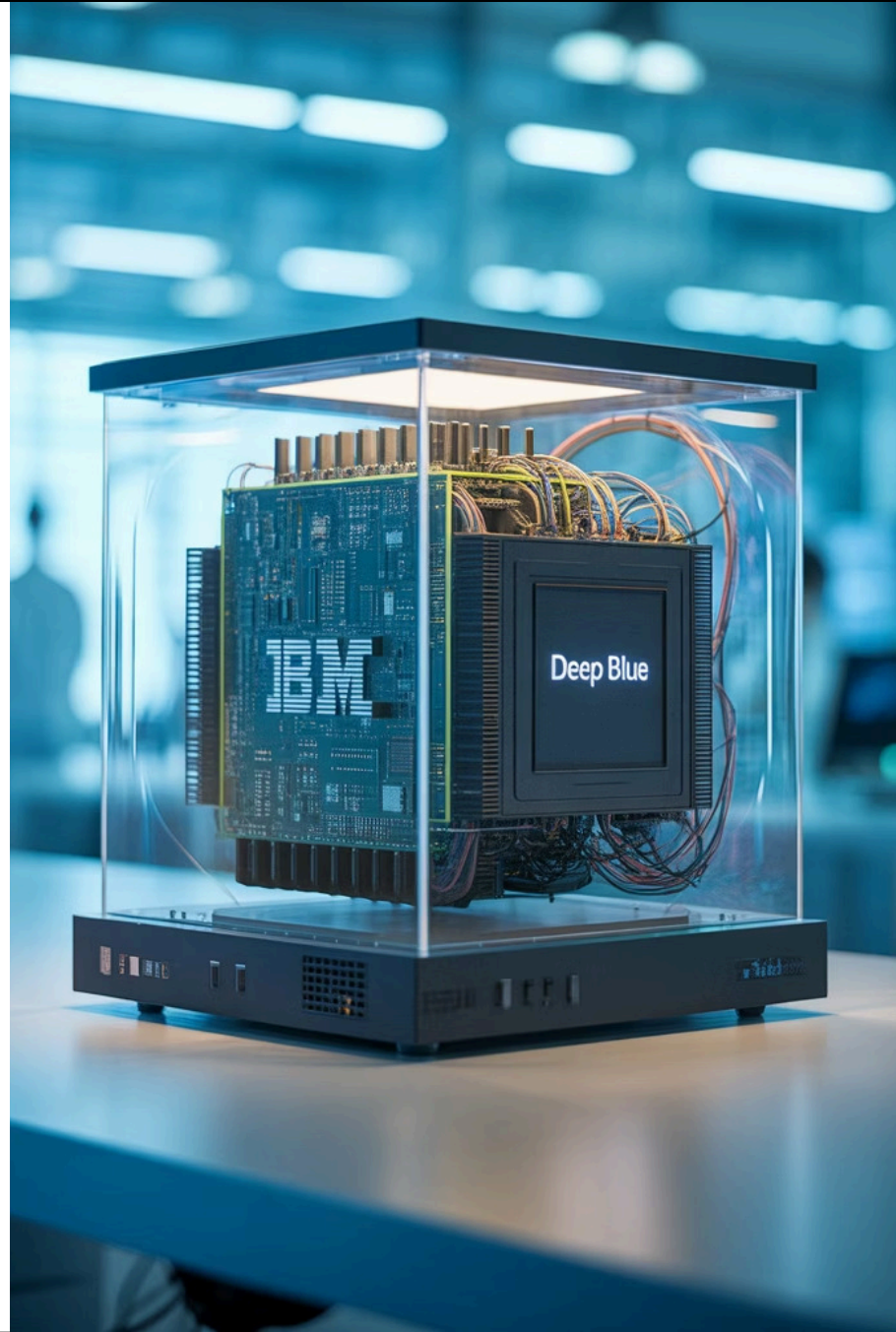
✅ **Why It Worked:** Brute-force + human-tuned heuristics = victory over Kasparov.



♟️ Slide 2: Why Deep Blue Stopped Playing

- ⚙️ **Specialized Machine:** Built only for chess (not general AI).
- 💼 **IBM's Goal:** Publicity & stock boost, not a lasting chess program.
- ⚔️ **Kasparov Dispute:** Claimed unfair play, demanded rematch → IBM declined.
- ⏩ **Fast Progress:** Chess engines on normal PCs (Fritz, Stockfish) soon surpassed it.

✅ **Summary:**
Deep Blue was a **one-time project** for IBM's showcase.
After Kasparov, it was retired — overtaken by modern engines on standard hardware.



Summary: Mastering Adversarial Search 🎓

Game Formulation

Games provide perfect testbeds for AI algorithms. They combine clear rules, defined objectives, and computational challenges that mirror real-world adversarial scenarios.

Minimax Algorithm

The theoretical foundation for optimal play in two-player zero-sum games. By assuming perfect play from both sides, minimax guarantees mathematically optimal decisions.

Alpha-Beta Pruning

The efficiency breakthrough that makes minimax practical. Through intelligent branch elimination, it achieves identical results while examining exponentially fewer nodes.

> 🏆 **Key Insight:** The combination of rigorous mathematical foundations (minimax) with clever optimizations (alpha-beta) creates algorithms that can compete with and surpass human experts in strategic games!

🧠 Theoretical Foundation

Minimax provides the mathematical guarantee of optimal play

⚡ Practical Efficiency

Alpha-beta makes these algorithms computationally feasible

🚀 Real-World Impact

These techniques power modern game-playing AI systems

You now understand the core algorithms that enabled computers to master games from tic-tac-toe to chess! These principles extend far beyond games into any scenario involving strategic decision-making against intelligent opponents. 🎯✨